

UNIVERZITET U BEOGRADU – ELEKTROTEHNIČKI
FAKULTET
KATEDRA ZA ELEKTRONIKU

DIGITALNA ELEKTRONIKA 1

Materijali za računske vežbe

**SINTEZA KOMBINACIONIH MREŽA POMOĆU KOLA
SREDNJEG STEPENA INTEGRACIJE**

Vežbe 2

Autori:

Nikola Petrović p.z.nikola@etf.bg.ac.rs
Haris Turkmanović haris@etf.bg.ac.rs

Beograd, 9. oktobar 2026.

Sadržaj

1	Zadaci	2
1.1	Zadatak 1	2
1.2	Zadatak 2	5
1.3	Zadatak 3	9
1.4	Zadatak 4	16
2	Zadaci za samostalni rad	31
2.1	Zadatak 1	31
2.2	Zadatak 2	31

1 Zadaci

1.1 Zadatak 1

- Predstaviti logičku šemu multipleksera 4/1
- Pomoću multipleksera 4/1 iz tačke a) i NE, ILI i I logičkih kola realizovati kombinacionu mrežu koja generiše izlaze Y_0 i Y_1 definisane logičkim funkcijama predstavljenim u tabeli 1.

Tabela 1: Tabela logičkih funkcija za zadatak 1.1

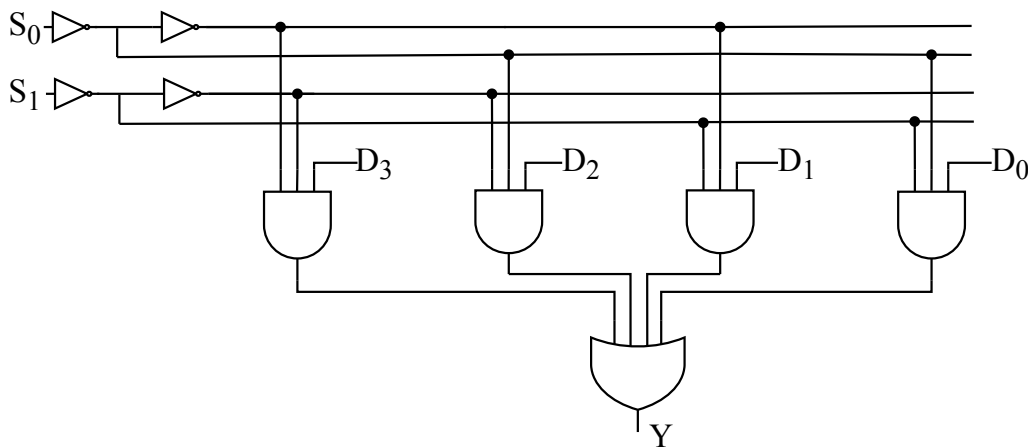
C_1C_0	Y_1	Y_0
00	$\overline{A \oplus B}$	$A \oplus B$
01	$A \oplus B$	$\overline{A \oplus B}$
10	$\bar{A}\bar{B}$	$\bar{A}B$
11	AB	$A\bar{B}$

Rešenje:

- Izlaz Y kombinacione mreže multipleksera 4/1, korišćenog za realizaciju u tački b) je definisan sledećom funkcijom:

$$Y = \bar{S}_1\bar{S}_0D_0 + \bar{S}_1S_0D_1 + S_1\bar{S}_0D_2 + S_1S_0D_3 \quad (1.1)$$

Na osnovu ove funkcije dobijamo realizaciju prikazanu na slici 1.



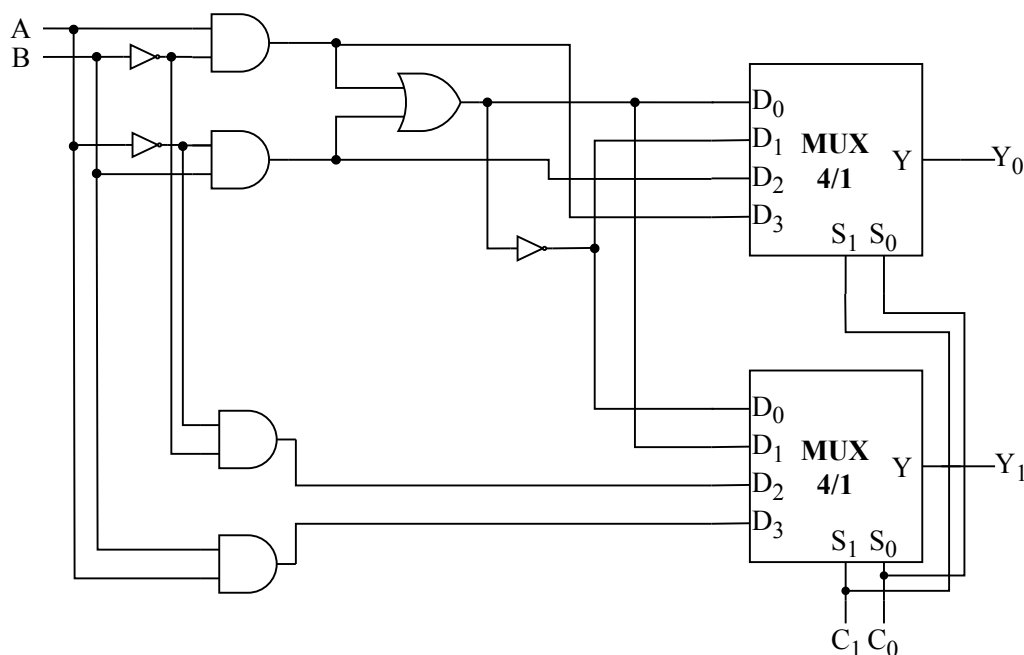
Slika 1: Implementacija multipleksera 4/1.

-
- Izlaze kombinacione mreže realizovaćemo korišćenjem dva multipleksera 4/1. Ukoliko funkcije koje se dovode na ulaz multipleksera nisu izražene u formi pogodnoj za realizaciju I, ILI ili NE logičkim kolima, najpre je neophodno izvesti njihovu odgovarajuću formu. Za slučaj $\overline{A \oplus B}$ i $A \oplus B$ dobijamo ekvivalentne izraze:

$$A \oplus B = A\bar{B} + \bar{A}B \quad (1.2)$$

$$\overline{A \oplus B} = \overline{A\bar{B} + \bar{A}B} \quad (1.3)$$

Nakon izvođenja ekvivalentnih izraza možemo nacrtati kombinacionu mrežu čiji izlazi su definisani funkcijama predstavljenim u tabeli 1. Ta kombinaciona mreža je predstavljena na slici 2.



Slika 2: Realizacija kombinacione mreže za zadatak 1.1.

Primeri su napisani u SystemVerilogu (fajlovi `.sv`) i simuliraju se Verilatorom. Verilog kod koji implementira multiplekser dat jednačinom (1.1).

```

module zadatak (
    input logic [1:0] S,
    input logic [3:0] D,
    output logic Y

);
    //  $Y = S_1' S_0' D_0 + S_1' S_0 D_1 + S_1 S_0' D_2 + S_1 S_0 D_3$ 
    assign Y = (~S[1] & ~S[0] & D[0]) |
        (~S[1] & S[0] & D[1]) |
        ( S[1] & ~S[0] & D[2]) |
        ( S[1] & S[0] & D[3]);
endmodule

```

Testbench koji simulira multiplekser dat jednačinom (1.1) je dat [OVDE](#).

Verilog kod koji implementira funkciju datu u tabeli 1.

```
module zadatak (  
    input logic A, B,  
    input logic [1:0] C,  
    output logic [1:0] Y  
);  
    logic [3:0] D_Y1, D_Y0;  
    logic axb;  
    assign axb = (A & ~B) | (~A & B);  
    // Ulazi MUX 0 za C = 00, 01, 10, 11.  
    assign D_Y0[0] = axb;  
    assign D_Y0[1] = ~axb;  
    assign D_Y0[2] = ~A & B;  
    assign D_Y0[3] = A & ~B;  
    // Ulazi MUX 1 za C = 00, 01, 10, 11.  
    assign D_Y1[0] = ~axb;  
    assign D_Y1[1] = axb;  
    assign D_Y1[2] = ~A & ~B;  
    assign D_Y1[3] = A & B;  
    mux4 mux_Y1 (.S(C), .D(D_Y1), .Y(Y[1]));  
    mux4 mux_Y0 (.S(C), .D(D_Y0), .Y(Y[0]));  
endmodule
```

Testbench koji simulira funkciju datu u tabeli 1 je dat [OVDE](#).

1.2 Zadatak 2

- a) Pomoću jednog multipleksera 4/1 i potrebnog broja logičkih kola projektovati kolo koje generiše izlazni signal Y definisan logičkom funkcijom:

$$Y = CB\bar{A} + CA + \bar{B}A \quad (1.4)$$

- b) Projektovati kolo u minimalnoj formi ako su dostupna isključivo dvoulazna NI kola.
 c) Uporediti realizaciju pod a) i pod b) u pogledu složenosti i kašnjenja ako je vreme propagacije signala kroz multiplekser $t_{d-mux} = 20\text{ns}$ a kroz logička kola $t_{d-l} = 10\text{ns}$.
 d) Da li se u realizaciji pod b) javljaju lažne nule, i ukoliko se javljaju jasno naznačiti pri kojim prelazima. Predstaviti realizaciju, u vidu logičke funkcije, kojom se otklanjaju lažne nule.

Rešenje:

a) Selekcione ulaze biramo poređenjem kofaktora funkcije za moguće parove promenljivih. Prime-nom Šenonovog razvoja svaki kofaktor postaje jedan podatkovni ulaz multipleksera. Povoljan je izbor koji zahteva najmanje dodatnih kola; broj pojavljivanja promenljivih u početnom zapisu sam po sebi ne garantuje minimum. Ovde uzimamo $S_1 S_0 = AB$: svi kofaktori biće konstante ili neposredno signal C .

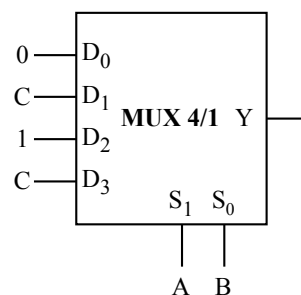
Logičku funkciju odgovarajućeg ulaza D_i multipleksera 4/1 određujemo primenjujući proceduru koja podrazumeva zamenu promenljivih AB u jednačini:

$$Y(A, B, C) = CB\bar{A} + CA + \bar{B}A \quad (1.5)$$

sa binarnim kombinacijama od 00 do 11:

$$\begin{aligned} Y(0, 0, C) &= 0 \\ Y(0, 1, C) &= C \\ Y(1, 0, C) &= 1 \\ Y(1, 1, C) &= C \end{aligned} \quad (1.6)$$

Na osnovu niza jednakosti (1.6) dobijamo realizaciju prikazanu na slici 3.



Slika 3: Realizacija funkcije (1.5) korišćenjem isključivo multipleksera 4/1 pri čemu se na selekcione signale multipleksera dovode signali A i B .

Za samostalni rad: Implementirati logičku funkciju Y ukoliko su kao selekcioni signali uzeti ulazi AC .

Verilog kod koji implementira kolo sa slike 3.

Napomena: Kašnjenja dodata u kod se koriste samo za simulacije!

```
module zadatak #(parameter time T = 20ns) (
    input logic A, B, C,
    output logic Y
);
    logic [1:0] S;
    logic [3:0] D;
    assign S[1] = A;
    assign S[0] = B;
    assign D = {C, 1'b1, C, 1'b0};
    // Prosledi parametar spoljasnjeg kola.
    mux4 #(.T(T)) UMUX (.S(S), .D(D), .Y(Y));
endmodule
```

Testbench koji simulira kolo sa slike 3 je dat OVDE.

b) Kao i do sada, prvi korak na putu do minimalne realizacije podrazumeva popunjavanje odgovarajuće Karnoove karte na osnovu koje se izvodi funkcija u minimalnoj formi. Sadržaj Karnoove karte za funkciju (1.4) je prikazan na slici 4.

BA \ C	00	01	11	10
0	0	1	0	0
1	0	1	1	1

Slika 4: Karnoova karta za funkciju (1.4).

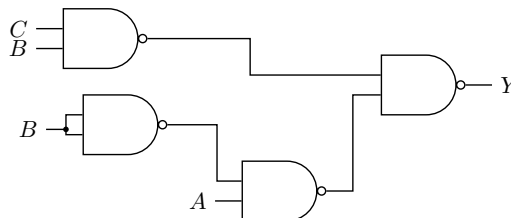
Za realizaciju korišćenjem isključivo dvoulaznih NI kola, u Karnoovoj karti tražimo oblasti koje sadrže logičke jedinice, tj. glavni cilj je realizacija funkcije (1.4) u formi zbira proizvoda. Na osnovu Karnoove karte sa slike 4, dobijamo sledeću funkciju:

$$Y = CB + \bar{B}A \quad (1.7)$$

Primenom pravila Bulove algebre $Y = \bar{\bar{Y}}$, dobijamo:

$$Y = \overline{\overline{CB + \bar{B}A}} = \overline{\overline{CB} \cdot \overline{\bar{B}A}} \quad (1.8)$$

Na osnovu (1.8) moguće je realizovati logičku funkciju Y kao što je prikazano na slici 5.



Slika 5: Realizacija logičke funkcije (1.7) pomoću NI kola.

Verilog kod koji implementira kolo sa slike 5.

Napomena: Kašnjenja dodata u kod se koriste samo za simulacije!

```

module zadatak #(parameter time T = 10ns) (
    input logic A,
    input logic B,
    input logic C,
    output logic Y
);
    logic I1, I2, I3;
    // I1 = ¬(C · B)
    assign #(T) I1 = ~(C & B);
    // I2 = ¬(B · B)
    assign #(T) I2 = ~(B & B);
    // I3 = ¬(A · I2)
    assign #(T) I3 = ~(A & I2);
    // Y = ¬(I1 · I3)
    assign #(T) Y = ~(I1 & I3);
endmodule

```

Testbench koji simulira kolo sa slike 5 je dat [OVDE](#).

c) Broj kućišta zavisi od izabrane biblioteke: uz četverostruko NI2 kolo (npr. 74HC00) i kućište koje sadrži multiplexer 4/1 (npr. 74HC153), obe realizacije staju u po jedno kućište. To nisu ista kola niti nužno isti broj priključaka: navedeni primeri imaju 14, odnosno 16 priključaka. NI realizacija sadrži četiri dvoulazna NI kola, uključujući invertovanje B vezivanjem dva ulaza istog kola. Iscrpno poređenje acikličnih mreža sa najviše tri NI2 kola, bez unapred dostupnih komplementarnih ulaza, ne daje ovu funkciju.

Za zadati model kašnjenja multiplexer daje $t_{d,Y} = t_{d,\text{mux}} = 20$ ns. Najduža putanja NI realizacije polazi od B i prolazi kroz tri kola, pa je najgore kašnjenje $3t_{d,l} = 30$ ns; putanje od A i C prolaze kroz dva kola. Ove vrednosti su pretpostavke zadatka, a ne katalogske vrednosti navedenih integrisanih kola.

d) Mogućnost lažne nule postoji kada je $A = C = 1$, a menja se B : susedna polja $CBA = 101$ i 111 nemaju zajedničku oblast. Oba smeru označena su strelicama na slici 6. Pri jednakim kašnjenjima T svih NI kola, za $B : 1 \rightarrow 0$ izlaz pada posle $2T$ i vraća se posle $3T$. Za obrnuti smer u tom modelu nema impulsa; drugačiji odnosi kašnjenja zahtevaju zasebnu analizu. Strelice označavaju strukturno rizičnu susednost, a ne garanciju impulsa pri svakom modelu kašnjenja.

$\begin{smallmatrix} B \\ A \end{smallmatrix}$	00	01	11	10
0	0	1	0	0
1	0	1	1	1

Slika 6: Karnoova karta funkcije (1.4) sa naznačenim prelazima koji dovode do pojave lažne nule.

U cilju sprečavanja pojave lažne nule, uvodimo dodatnu oblast kao što je to prikazano na slici 7 i dobijamo funkciju:

$$Y = CB + \bar{B}A + CA \quad (1.9)$$

C \ BA		00	01	11	10
		0	1	0	0
0		0	1	0	0
1		0	1	1	1

Slika 7: Karnoova karta za funkciju (1.4).

Verilog kod koji implementira jednačinu (1.9). Svaki proizvod modelovan je jednom dodelom sa inertnim kašnjenjem T ; zasebno kašnjenje invertora B ovde nije uvedeno. Član CA pokriva rizičnu susednost i sprečava lažnu nulu pri promeni samo jednog ulaza.

Napomena: Kašnjenja dodata u kod se koriste samo za simulacije!

```

module zadatak #(parameter time T = 10ns) (
    input logic A,
    input logic B,
    input logic C,
    output logic Y
);
    logic I1, I2, I3;
    assign #(T) I1 = (C & B); // I1 = C · B
    assign #(T) I2 = (~B & A); // I2 = ¬B · A
    assign #(T) I3 = (C & A); // I3 = C · A
    assign #(T) Y = (I1 | I2 | I3); // Y = I1 + I2 + I3
endmodule

```

Testbench koji simulira jednačinu (1.9) je dat [OVDE](#).

1.3 Zadatak 3

- Izvesti funkciju svakog izlaza dekodera 3/8. Izlazi dekodera su aktivni u logičkoj jedinici i dekodер ne sadrži *enable* ulaze
- Pomoću dekodera 3/8 sa *enable* ulazom EN_0 aktivnim na nivou logičke jedinice i EN_1 i EN_2 aktivnim na nivou logičke nule, projektovati dekodер 4/16. Izlazi dekodera 3/8 su aktivni na nivou logičke nule, dok su ostali ulazi (osim EN_1 i EN_2) aktivni na nivou logičke jedinice. Izlazi dekodera 4/16 takođe treba da budu aktivni na nivou logičke nule.
- Pomoću dekodera 3/8 iz prethodne tačke, i dvoulaznih I logičkih kola, realizovati sledeću funkciju:

$$Y = \bar{B}A + C\bar{B}\bar{A} + \bar{C}BA \quad (1.10)$$

- Pomoću dekodera iz tačke a) realizovati dekodер 6/64

Rešenje:

- Tabela 2 predstavlja funkcionalnu tabelu kojom se opisuje način rada dekodera 3/8.

Tabela 2: Funkcionalna tabela dekodera 3/8 u opštem slučaju

$A_2A_1A_0$	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
000	0	0	0	0	0	0	0	1
001	0	0	0	0	0	0	1	0
010	0	0	0	0	0	1	0	0
011	0	0	0	0	1	0	0	0
100	0	0	0	1	0	0	0	0
101	0	0	1	0	0	0	0	0
110	0	1	0	0	0	0	0	0
111	1	0	0	0	0	0	0	0

Na osnovu funkcionalne tabele 2 možemo izvesti logičku funkciju svakog od izlaza:

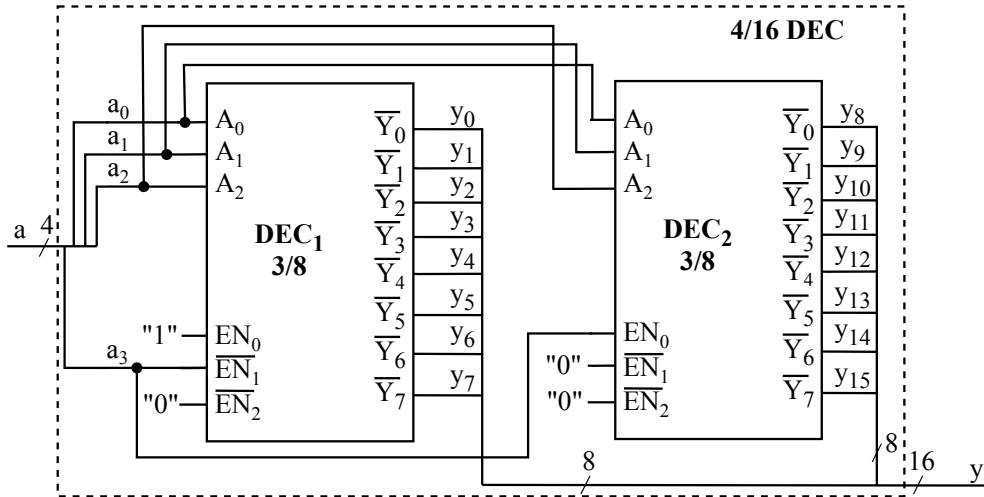
$$\begin{aligned}
 Y_0 &= \bar{A}_2\bar{A}_1\bar{A}_0 \\
 Y_1 &= \bar{A}_2\bar{A}_1A_0 \\
 Y_2 &= \bar{A}_2A_1\bar{A}_0 \\
 Y_3 &= \bar{A}_2A_1A_0 \\
 Y_4 &= A_2\bar{A}_1\bar{A}_0 \\
 Y_5 &= A_2\bar{A}_1A_0 \\
 Y_6 &= A_2A_1\bar{A}_0 \\
 Y_7 &= A_2A_1A_0
 \end{aligned} \quad (1.11)$$

Verilog kod koji implementira dekodera opisan tabelom 2.

```
module zadatak (  
    input logic [2:0] A,  
    output logic [7:0] Y  
);  
    always_comb begin  
        case (A)  
            3'b000: Y = 8'b00000001; // Y0  
            3'b001: Y = 8'b00000010; // Y1  
            3'b010: Y = 8'b00000100; // Y2  
            3'b011: Y = 8'b00001000; // Y3  
            3'b100: Y = 8'b00010000; // Y4  
            3'b101: Y = 8'b00100000; // Y5  
            3'b110: Y = 8'b01000000; // Y6  
            3'b111: Y = 8'b10000000; // Y7  
            default: Y = 'x; // Nebinarna adresa.  
        endcase  
    end  
endmodule
```

Testbench koji simulira dekodera opisan tabelom 2 je dat [OVDE](#).

b) U ovoj tački se zahteva realizacija dekodera 4/16 korišćenjem dekodera 3/8. Za razliku od dekodera 3/8 iz prethodne tačke, ovaj dekodер sadrži tri ulazna enable signala dok su mu izlazi aktivni u logičkoj nuli. Ako je $E = \overline{EN_0} \overline{EN_1} \overline{EN_2} = 1$, izlazi predstavljaju komplemente funkcija iz (1.11). Opšti izraz je $Y_{i,akt.0} = \overline{E}Y_i$: kada je $E = 0$, svi izlazi su jedinice. Za $A_3 = 0$ omogućen je donji skup izlaza 0–7, a za $A_3 = 1$ gornji skup 8–15. Magistrala od 16 bita na slici objedinjuje dva različita skupa vodova; ne spaja izlaze međusobno.



Slika 8: Realizacija dekodera 4/16 korišćenjem dva dekodera 3/8.

Verilog kod koji implementira kolo sa slike 8.

```
module zadatak (
    input logic [3:0] A,
    output logic [15:0] Y
);
    logic [7:0] Y_low, Y_high;
    // Dekoder za donjih osam izlaza.
    decoder dec_low (.EN0(1'b1), .EN1_B(A[3]), .EN2_B(1'b0),
        .A(A[2:0]), .Y(Y_low));
    // Dekoder za gornjih osam izlaza.
    decoder dec_high (.EN0(A[3]), .EN1_B(1'b0), .EN2_B(1'b0),
        .A(A[2:0]), .Y(Y_high));
    assign Y[7:0] = Y_low;
    assign Y[15:8] = Y_high;
endmodule
```

Testbench koji simulira kolo sa slike 8 je dat OVDE.

c) Pošto se u ovoj tački zahteva realizacija funkcije (1.10) korišćenjem dekodera, navedenu funkciju je neophodno izraziti u formi koja sadrži neke od komplementiranih proizvoda (1.11). Do ove forme je najlakše doći analizirajući Karnoovu kartu koja odgovara funkciji (1.10) i izdvajanjem izlaza aktivnih u logičkoj nuli koji se formiraju potpunim zbirovima. Sadržaj Karnoove karte, sa naznačenim oblastima koji se realizuju u formi potpunih zbirova je prikazan na slici 9.

Na osnovu oblasti označenih na slici 9, izvodimo sledeću formu funkcije (1.10):

$$Y = (C + B + A)(C + \bar{B} + A)(\bar{C} + \bar{B} + \bar{A}) \quad (1.12)$$

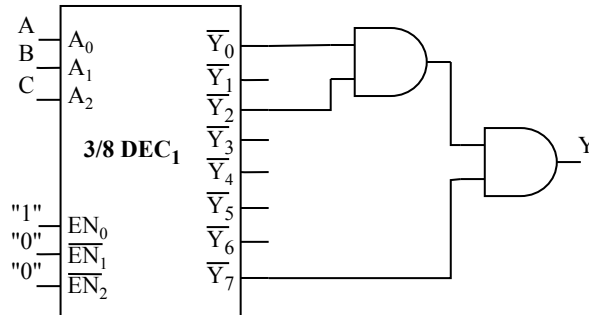
Ukoliko funkciju (1.12) izrazimo koristeći funkcije date jednačinom (1.11) dobijamo:

$$Y = \bar{Y}_0 \bar{Y}_2 \bar{Y}_7 \quad (1.13)$$

Na osnovu (1.13) realizujemo kombinacionu mrežu prikazanu na slici 10.

$\begin{array}{c} B \\ \backslash \\ C \end{array} A$	00	01	11	10
0	0	1	1	0
1	1	1	0	1

Slika 9: Karnoova karta za funkciju (1.10).



Slika 10: Kombinaciona mreža funkcije (1.10) realizovana korišćenjem dekodera 3/8.

Verilog kod koji implementira jednačinu (1.13) i kolo sa slike 10.

```

module zadatak (
    input logic A, B, C,
    output logic Y_comb, Y_dekoder
);
    logic [2:0] A_dec;
    logic [7:0] Y_dec;
    assign A_dec = {C, B, A};
    decoder udec (.EN0(1'b1), .EN1_B(1'b0), .EN2_B(1'b0),
        .A(A_dec), .Y(Y_dec));
    // Potpuni zbirovi i odgovarajući aktivno niski izlazi dekodera.
    assign Y_comb = (C | B | A) & (C | ~B | A) & (~C | ~B | ~A);
    assign Y_dekoder = Y_dec[0] & Y_dec[2] & Y_dec[7];
endmodule

```

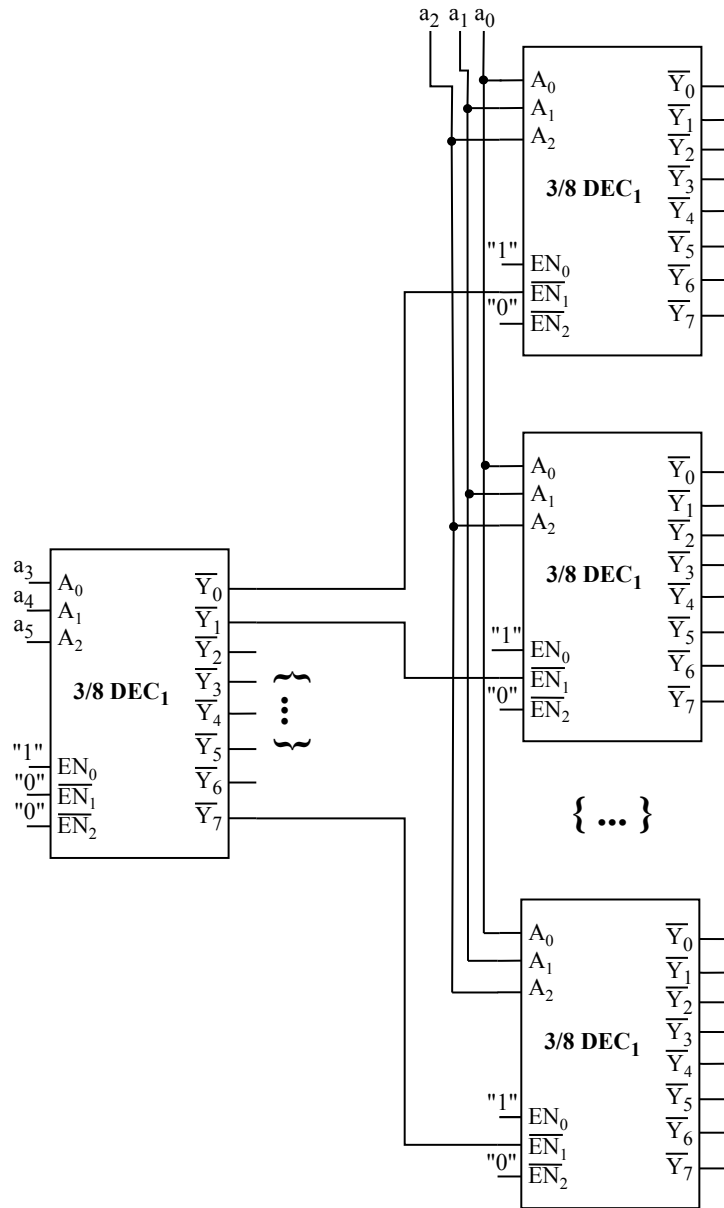
Testbench koji simulira jednačinu (1.13) i kolo sa slike 10 je dat OVDE.

d) Dekoder iz tačke a) nema ulaze za dozvolu rada. Zato se prikazana kaskada sa signalima *enable* ne može neposredno sastaviti od tih blokova. Za doslovnu postavku najpre dva dekodera iz a) dekodiraju $A_5A_4A_3$ i $A_2A_1A_0$ u signale H_i i L_j . Izlazi dekodera 6/64 aktivni u jedinici određeni su izrazom

$$Y_{8i+j} = H_i L_j, \quad i, j \in \{0, \dots, 7\}. \quad (1.14)$$

Potrebna su 64 dvoulazna I kola. Ako su dozvoljeni isključivo dekoderi iz a), svaki proizvod pq može se dobiti sa izlaza Y_3 dodatnog dekodera čiji su ulazi $A_2A_1A_0 = 0pq$; time se dobija realizacija sa 66 dekodera. Ne tvrdi se da je ovaj broj minimalan.

Šema na slici 11 predstavlja alternativnu realizaciju koja koristi devet dekodera sa ulazima za dozvolu i izlazima aktivnim u nuli iz tačke b). Prvi dekodeer aktivira tačno jedan od osam dekodera drugog stepena preko ulaza $\overline{EN}_1$. Izlaz j grane i predstavlja globalni izlaz $8i + j$, aktivan u nuli. Ova varijanta zahteva drugačiji tip dekodera od onog navedenog u tački d).



Slika 11: Alternativna realizacija dekodera 6/64 pomoću dekodera iz tačke b), sa izlazima aktivnim u nuli.

U narednim primerima indeks $8i + j$ povezuje adresu gornjeg i donjeg dekodera sa globalnim

izlazom. Zapis **generate** ponavlja opis istog kola: pri elaboraciji nastaje zasebno kolo ili instanca za svaku vrednost indeksa, a ne petlja koja se izvršava tokom simulacije. Verilog kod: dva dekodera 3/8 i 64 I kola, prema jednačini (1.14). Izlazi su aktivni u jedinici. Ova varijanta zahteva I kola pored dekodera iz tačke a).

```
module zadatak (
    input logic [5:0] A,
    output logic [63:0] Y
);
    logic [7:0] H, L;
    decoder3 HIGH (.A(A[5:3]), .Y(H));
    decoder3 LOW (.A(A[2:0]), .Y(L));
    // Svaka iteracija stvara jedno zasebno dvoulazno I kolo.
    generate
        for (genvar i = 0; i < 8; i++) begin : grana
            for (genvar j = 0; j < 8; j++) begin : izlaz
                assign Y[8*i+j] = H[i] & L[j];
            end
        end
    endgenerate
endmodule
```

Pomoćni moduli: [decoder3.sv](#).

Testbench i automatske provere: [tb_zadatak.sv](#).

Verilog kod: realizacija sa 66 dekodera iz tačke a), bez dodatnih I kola i bez ulaza za dozvolu rada. Izlaz 3 pomoćnog dekodera sa adresom $0H_iL_j$ daje proizvod H_iL_j , pa se doslovno poštuje zadati skup blokova.

```
module zadatak (
    input logic [5:0] A,
    output logic [63:0] Y
);
    logic [7:0] H, L;
    decoder3 HIGH (.A(A[5:3]), .Y(H));
    decoder3 LOW (.A(A[2:0]), .Y(L));
    // Y3 dekodera sa adresom Opq predstavlja proizvod pq.
    generate
        for (genvar i = 0; i < 8; i++) begin : grana
            for (genvar j = 0; j < 8; j++) begin : izlaz
                logic [7:0] P;
                decoder3 PRODUCT (.A({1'b0, H[i], L[j]}), .Y(P));
                assign Y[8*i+j] = P[3];
            end
        end
    endgenerate
endmodule
```

Pomoćni moduli: [decoder3.sv](#).

Testbench i automatske provere: [tb_zadatak.sv](#).

Verilog kod: alternativna kaskada sa slike 11. Koristi devet dekodera iz tačke b), sa dozvolom rada i izlazima aktivnim u nuli; nisu upotrebljeni dekoderi iz tačke a).

```
module zadatak (  
    input logic [5:0] A,  
    output logic [63:0] Y  
);  
    logic [7:0] H;  
    // Prvi stepen bira jedan od osam aktivno niskih EN ulaza.  
    decoder HIGH (.EN0(1'b1), .EN1_B(1'b0), .EN2_B(1'b0),  
        .A(A[5:3]), .Y(H));  
    generate  
        for (genvar i = 0; i < 8; i++) begin : grana  
            decoder LOW (.EN0(1'b1), .EN1_B(H[i]), .EN2_B(1'b0),  
                .A(A[2:0]), .Y(Y[8*i+:8]));  
        end  
    endgenerate  
endmodule
```

Pomoćni moduli: [decoder.sv](#).

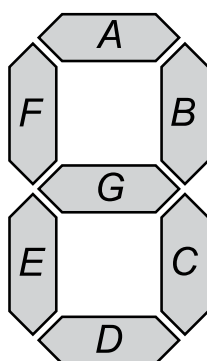
Testbench i automatske provere: [tb_zadatak.sv](#).

1.4 Zadatak 4

- Projektovati konvertor BCD koda u sedmobitni kod za pobudu svetlosnog sedmosegmentnog LED indikatora sa zajedničkom katodom. Ako se na ulazu pojave kombinacije od 0000 do 1001 na indikatoru se prikazuju cifre 0-9. Ako se na ulazu pojave nedozvoljene kombinacije 1010-1111, na indikatoru se prikazuje slovo E (*Error*). Na raspolaganju su samo NI i NILI logička kola. Težiti da broj upotrebljenih logičkih kola bude minimalan.
- Projektovati kombinacionu mrežu, ako je poznato da se na ulazu konvertora koda ne mogu pojaviti nedozvoljene kombinacije ulaza, tj. ulazni podatak je sigurno u opsegu 0000–1001. Na raspolaganju su logička kola proizvoljnog tipa.
- Kako treba modifikovati dobijenu šemu pod tačkom a) da bi se u višecifrenom indikatoru gasile vodeće nule i prikazivalo samo jedno slovo E kada se na ulazu bilo koje cifre pojavi nedozvoljena kombinacija? Na raspolaganju su logička kola proizvoljnog tipa. Projektovati kombinacionu mrežu bez povratnih putanja, tako da prikaz zavisi samo od trenutnih ulaza. Za broj 0000 prikazati jednu nulu, a slovo E prikazati na mestu jedinica.

Tabela 3: Primer rada višecifrenog BCD konvertora

Ulaz u višecifreni BCD konvertor	Sadržaj na višecifrenom sedmosegmentnom LED displeju
0231	231
00A5	E
0504	504



Slika 12: Sedmosegmentni displej sa označenim segmentima. Velika slova na slici predstavljaju segmente *a–g*, a ne ulazne bitove BCD koda.

Rešenje:

a) Najpre određujemo funkcionalnu tabelu koja definiše vrednosti izlaznih signala za sve kombinacije ulaznih logičkih nivoa. Na izlazu kombinacione mreže imamo sedmobitni kod (abcdefg) koji se koristi za pobudu LED segmenata. S obzirom na to da je, prema tekstu zadatka, indikator sa zajedničkom katodom, aktivan logički nivo za pobudu nekog segmenta indikatora je nivo logičke jedinice.

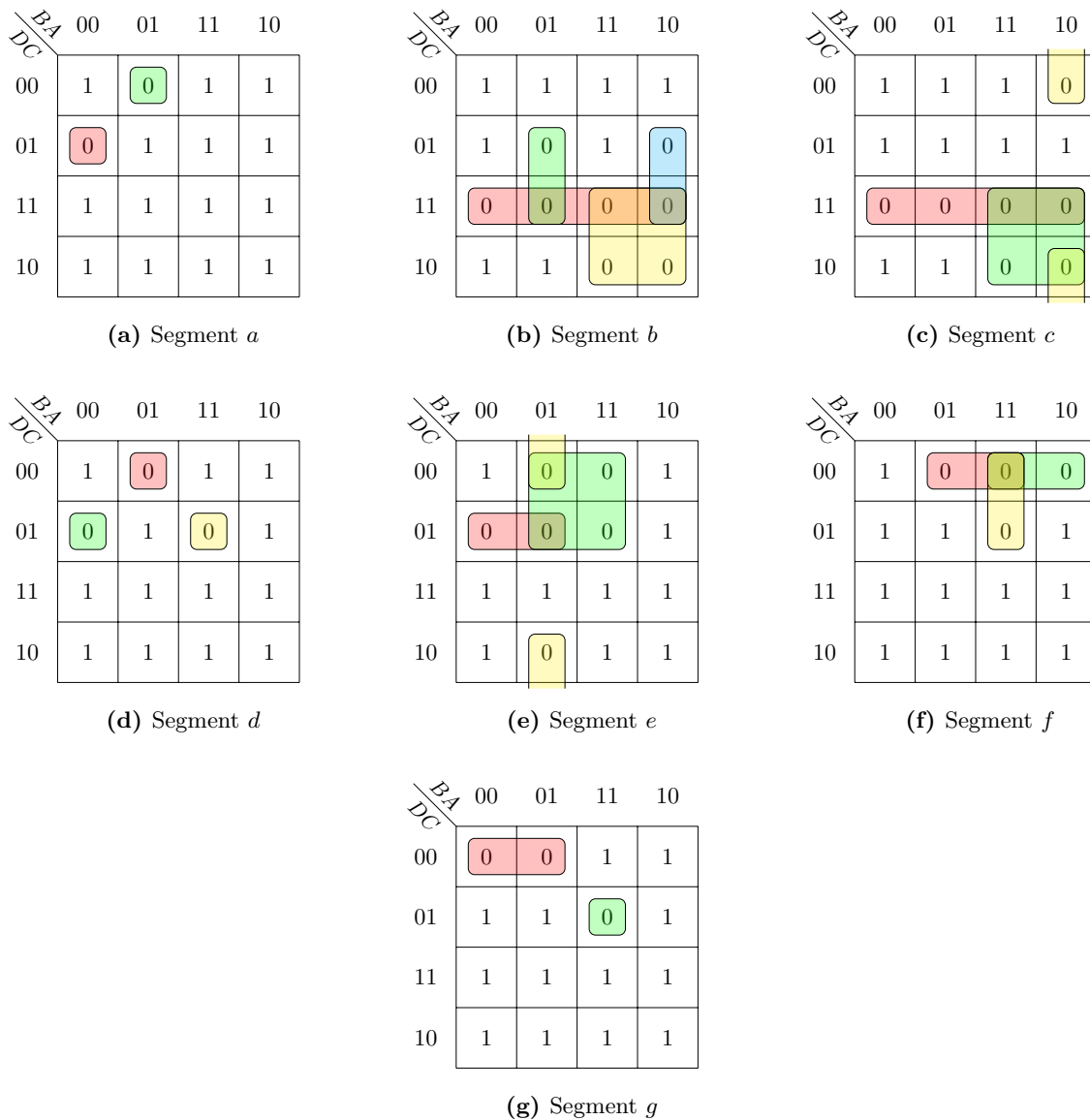
Tabela 4: Funkcionalna tabela uz zadatak 1.4

D	C	B	A	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1
1	0	1	0	1	0	0	1	1	1	1
1	0	1	1	1	0	0	1	1	1	1
1	1	0	0	1	0	0	1	1	1	1
1	1	0	1	1	0	0	1	1	1	1
1	1	1	0	1	0	0	1	1	1	1
1	1	1	1	1	0	0	1	1	1	1

Na osnovu kombinacione tabele 4, popunjavamo odgovarajuće Karnoove karte a zatim određujemo funkciju svakog od izlaza. U nastavku su date Karnoove karte za svaki od izlaza BCD konvertora.

Grupisanjem nula u Karnoovim kartama dobijaju se jednostavni proizvodi zbirova, pogodni za dvostepenu NILI realizaciju. Minimum broja članova i literala takvog zapisa nije opšti dokaz minimalnog broja kola u višestepenoj mreži, niti uključuje moguću zajedničku logiku različitih segmenata.

$$\begin{aligned}
 a &= (D + \bar{C} + B + A)(D + C + B + \bar{A}) \\
 b &= (\bar{D} + \bar{B})(\bar{D} + \bar{C})(\bar{C} + \bar{B} + A)(\bar{C} + B + \bar{A}) \\
 c &= (\bar{D} + \bar{B})(\bar{D} + \bar{C})(C + \bar{B} + A) \\
 d &= (D + \bar{C} + B + A)(D + C + B + \bar{A})(D + \bar{C} + \bar{B} + \bar{A}) \\
 e &= (D + \bar{A})(C + B + \bar{A})(D + \bar{C} + B) \\
 f &= (D + C + \bar{A})(D + C + \bar{B})(D + \bar{B} + \bar{A}) \\
 g &= (D + C + B)(D + \bar{C} + \bar{B} + \bar{A})
 \end{aligned} \tag{1.15}$$



Slika 13: Sadržaj Karnoovih karti za svaki od izlaza BCD konvertora.

Za samostalni rad: Izvesti funkciju za svaki od izlaza ako se umesto NILI logičkih kola koriste NI logička kola. Uporediti dobijene funkcije sa funkcijama datih u jednačini (1.15) sa stanovišta broja logičkih kola.

Primenom Bulovih transformacija nad funkcijom svakog od signala potencijalno je moguće dodatno uprostiti izraze. U slučaju funkcije izlaznog signala *a* važe sledeće jednakosti:

$$a = (D + \bar{C} + B + A)(D + C + B + \bar{A}) \quad (1.16)$$

primenom pravila Bulove algebre dobija se

$$\begin{aligned} a &= (D + \bar{C} + B + A)(D + C + B + \bar{A}) \\ &= D + B + (\bar{C} + A)(C + \bar{A}) = D + B + (\bar{C}\bar{A} + CA) \\ &= D + B + \overline{C \oplus A} \end{aligned} \quad (1.17)$$

Na osnovu dobijenog izraza (1.17) jasno je da se funkcija *a* može realizovati sa manjim brojem

logičkih kola. Međutim, za takvu realizaciju potrebno je koristiti EXNILI logička kola koja nisu dozvoljena u ovoj tački zadatka.

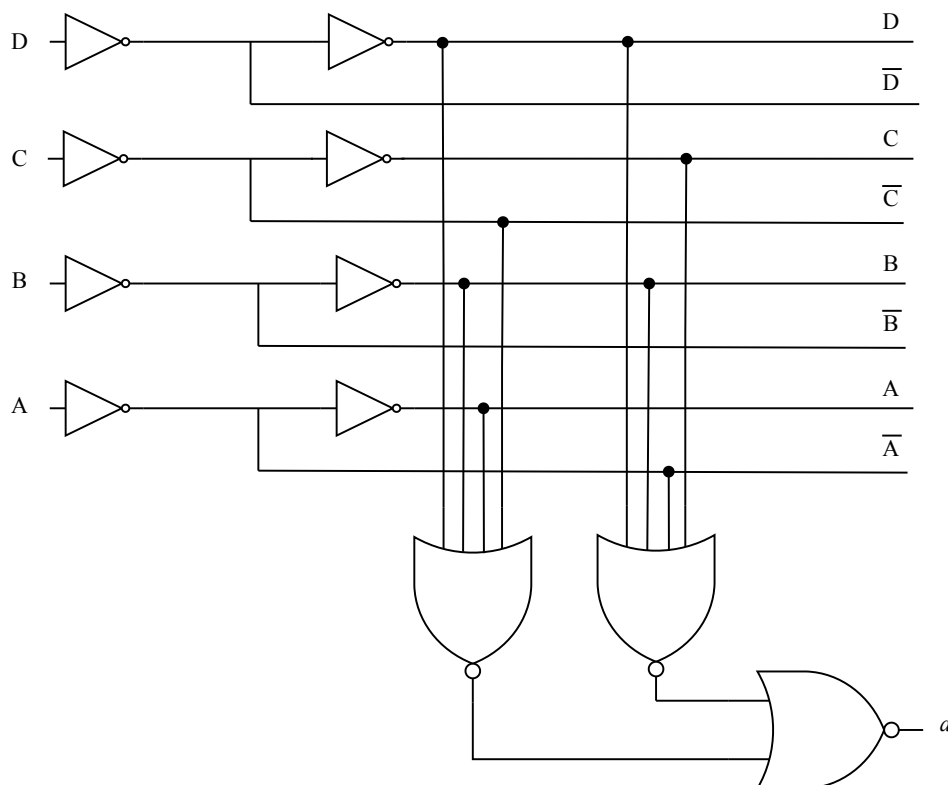
Za samostalni rad: Za ostale signale b-g, primenom Bulovih transformacija, naći formu koja sadrži manji broj logičkih kola

Pošto se u ovoj tački zadatka traži realizacija korišćenjem isključivo NI ili NILI logičkih kola, potrebno je funkcije izlaza iz Tabele napisati u odgovarajućoj formi. Pošto smo funkcije, u navedenoj tabeli, izrazili koristeći formu proizvoda zbirova, navedene funkcije je poželjno realizovati korišćenjem NILI logičkih kola. Zbog toga što broj ulaza nije ograničen tekstem zadatka, korišćemo NILI kola sa proizvoljnim brojem ulaza. Procedurom sličnom kao u prethodnim zadacima, dolazimo do forme logičke funkcije izlaznog signala a koja je pogodna za realizaciju NILI logičkim kolima:

$$\begin{aligned} a = \bar{\bar{a}} &= \overline{(D + \bar{C} + B + A)(D + C + B + \bar{A})} \\ &= \overline{(D + \bar{C} + B + A)} + \overline{(D + C + B + \bar{A})} \end{aligned} \quad (1.18)$$

Inverzije u razvodu ulaznih signala prikazane su uobičajenim trougaonim simbolom. Za ograničenje na NILI kola svaki takav invertor ostvaruje se NILI kolom sa spojenim ulazima; $\bar{X} + \bar{X} = \bar{X}$. Dodatni invertori koji ponovo stvaraju direktan signal služe preglednom razvodu i nisu obavezni u minimalnoj realizaciji.

Na slici 14 je ilustrovan implementacija izlaznog signala a korišćenjem isključivo NILI logičkih kola.



Slika 14: Implementacija izlaza a korišćenjem NILI logičkih kola.

Za samostalni rad:

1. Za ostale signale b-g izvesti algebarski zapis pogodan za implementaciju korišćenjem isključivo NILI logičkih kola.
2. Za ostale signale b-g izvesti algebarski zapis pogodan za implementaciju korišćenjem isključivo NI logičkih kola.
3. Uporediti realizacije sa stanovišta broja logičkih kola

Verilog kod koji implementira svih sedam segmenata prema jednačini (1.15) korišćenjem NILI kola. Svaki međusignal predstavlja izlaz jednog kola; zajednički članovi koriste se u više segmenata.

```

module zadatak (
    input logic D, C, B, A,
    output logic a, b, c, d, e, f, g
);
    logic D_n, C_n, B_n, A_n;
    logic a1, a2;
    logic b1, b2, b3, b4;
    logic c1, d1;
    logic e1, e2, e3;
    logic f1, f2, f3;
    logic g1;

    // Invertori: NILI kola sa vezanim ulazima.
    assign D_n = ~(D | D);
    assign C_n = ~(C | C);
    assign B_n = ~(B | B);
    assign A_n = ~(A | A);

    // Segment a.
    assign a1 = ~(D | C_n | B | A);
    assign a2 = ~(D | C | B | A_n);
    assign a = ~(a1 | a2);

    // Segment b; b1 i b2 koriste se i za segment c.
    assign b1 = ~(D_n | B_n);
    assign b2 = ~(D_n | C_n);
    assign b3 = ~(C_n | B_n | A);
    assign b4 = ~(C_n | B | A_n);
    assign b = ~(b1 | b2 | b3 | b4);

    // Segment c.
    assign c1 = ~(C | B_n | A);
    assign c = ~(b1 | b2 | c1);

    // Segment d; a1 i a2 su zajednicki sa segmentom a.
    assign d1 = ~(D | C_n | B_n | A_n);
    assign d = ~(a1 | a2 | d1);

    // Segment e.
    assign e1 = ~(D | A_n);
    assign e2 = ~(C | B | A_n);
    assign e3 = ~(D | C_n | B);
    assign e = ~(e1 | e2 | e3);

    // Segment f.
    assign f1 = ~(D | C | A_n);
    assign f2 = ~(D | C | B_n);
    assign f3 = ~(D | B_n | A_n);
    assign f = ~(f1 | f2 | f3);

    // Segment g; d1 je zajednicki sa segmentom d.
    assign g1 = ~(D | C | B);
    assign g = ~(g1 | d1);
endmodule

```

Testbench koji simulira konvertor dat jednačinom (1.15) je dat [OVDE](#).

b) Najpre određujemo kombinacionu tabelu koja definiše vrednosti izlaznih signala za sve kombinacije ulaznih logičkih nivoa. Na izlazu imamo sedmobitni binarni broj (abcdefg). Kako je poznato da na ulazu konvertora može biti samo neka od kombinacija 0000-1001, vrednosti izlaza, u slučaju nedozvoljene kombinacije signala na ulazu kola, mogu imati proizvoljnu vrednost (označenu sa bbbbbbb u funkcionalnoj tabeli) te ih je preporučljivo koristiti prilikom određivanja

jednostavnijeg zbira proizvoda. Oznaka b u ćeliji znači „bilo šta“ (don't care), a ne izlazni segment b . Takve ćelije smeju se koristiti samo uz navedenu garanciju da ulaz nikada nije 10–15; u varijanti a) izlazi za te ulaze moraju dati slovo E. Minimum broja članova i literala dvostepene forme ne garantuje minimum broja kola u svakoj biblioteci, naročito uz deljenje međurezultata između segmenata.

Tabela 5: Funkcionalna tabela uz zadatak 1.4 b).

D	C	B	A	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1
1	0	1	0	b	b	b	b	b	b	b
1	0	1	1	b	b	b	b	b	b	b
1	1	0	0	b	b	b	b	b	b	b
1	1	0	1	b	b	b	b	b	b	b
1	1	1	0	b	b	b	b	b	b	b
1	1	1	1	b	b	b	b	b	b	b

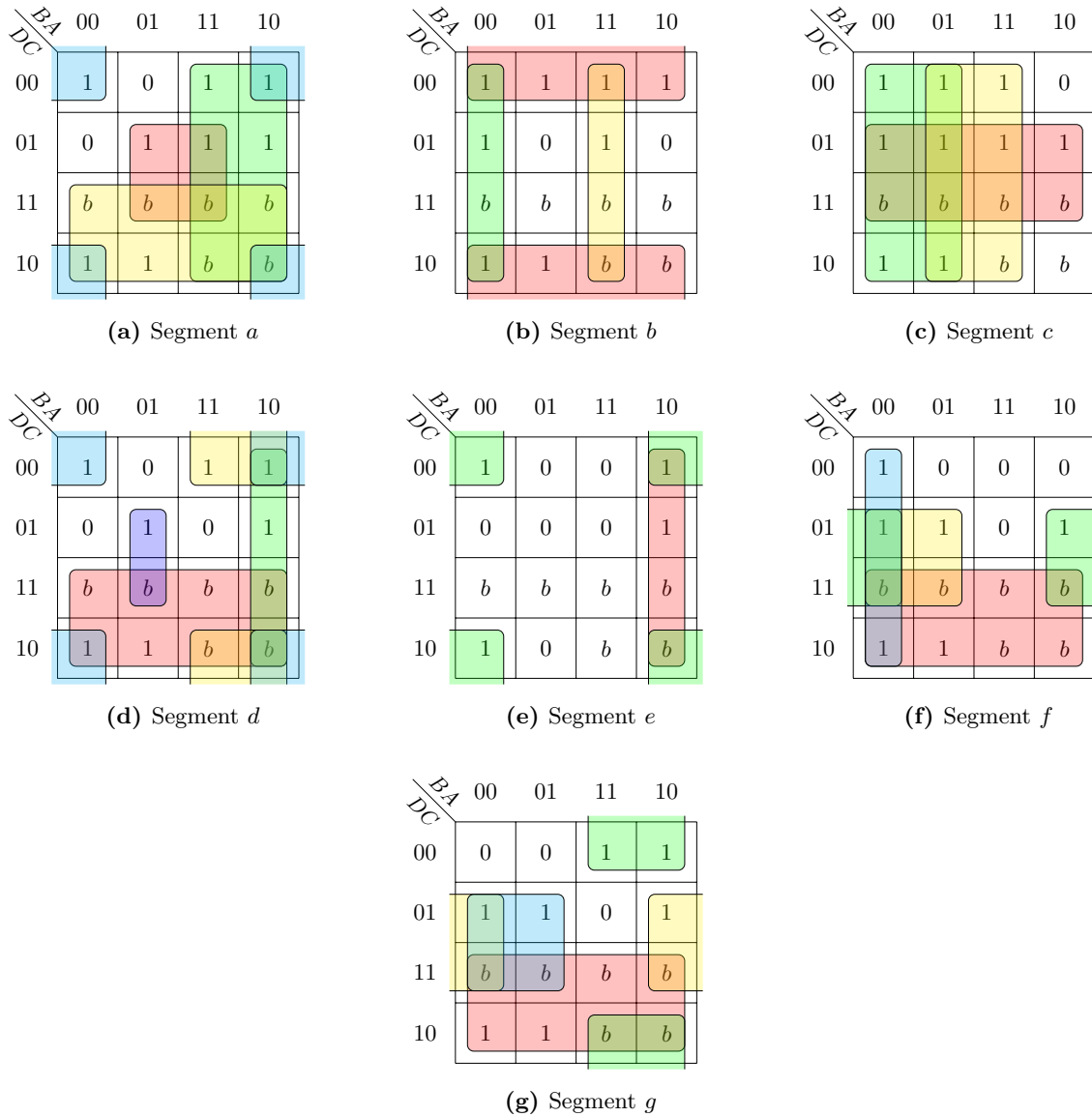
Na slici 15 prikazane su Karnoove karte gde su grupisane jedinice. Dobijene funkcije predstavljene u formi zbira proizvoda su date u okviru jednačine (1.19).

$$\begin{aligned}
 a &= D + B + CA + \bar{C}\bar{A} \\
 b &= \bar{C} + AB + \bar{A}\bar{B} \\
 c &= C + \bar{B} + A \\
 d &= D + B\bar{A} + B\bar{C} + \bar{C}\bar{A} + C\bar{B}A \\
 e &= B\bar{A} + \bar{C}\bar{A} \\
 f &= D + C\bar{A} + C\bar{B} + \bar{A}\bar{B} \\
 g &= D + B\bar{C} + C\bar{A} + C\bar{B}
 \end{aligned} \tag{1.19}$$

Posmatrajući Karnoovu kartu za izlaz a koja je data na slici 15a, izlaz a možemo dodatno pojednostaviti:

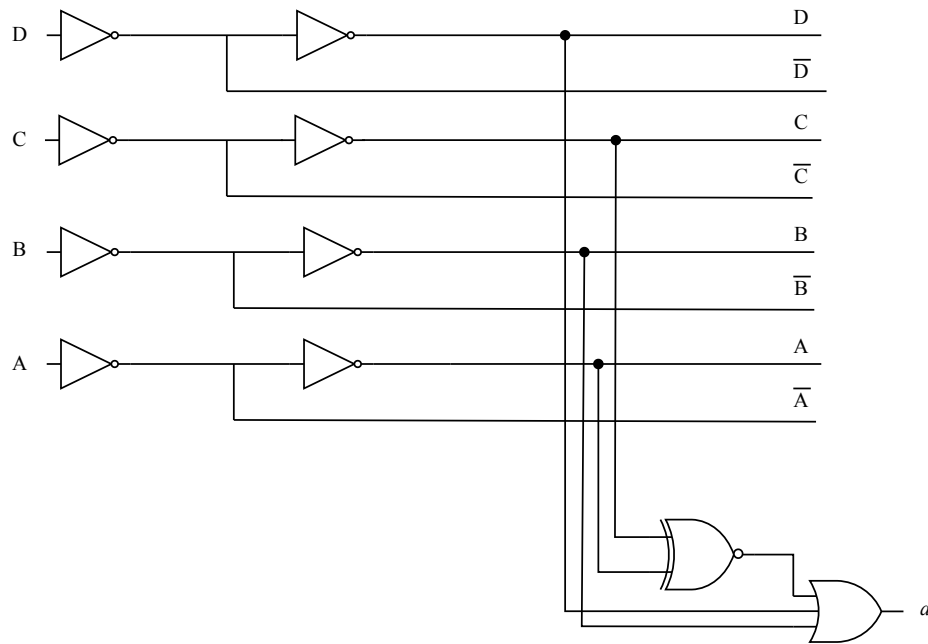
$$\begin{aligned}
 a &= D + B + CA + \bar{C}\bar{A} \\
 &= D + B + \overline{C \oplus A}
 \end{aligned} \tag{1.20}$$

Implementacija izlaza a datog jednačinom (1.20) je prikazana na slici 16.



Slika 15: Sadržaj Karnoovih karti za svaki od izlaza BCD konvertora za zadatak 1.4 b).

Za samostalni rad: Za ostale izlazne signale b-g, implementirati minimalne funkcije. Uporediti dobijene implementacije sa implementacijama u tački a).

Slika 16: Implementacija funkcije izlaza a .

Verilog kod koji implementira konvertor za dozvoljene BCD ulaze. Segment a prati šemu sa EXNILI kolom; ostali segmenti prate jednačinu (1.19). Za ulaze 10–15 prikaz nije zadat.

```

module zadatak (
    input logic D, C, B, A,
    output logic a, b, c, d, e, f, g
);
    // Ulaz je dozvoljena BCD cifra (0-9).
    // Segment a: EXNILI iz nacrtane realizacije.
    assign a = D | B | ~(C ^ A);

    // Segment b.
    assign b = ~C | (A & B) | (~A & ~B);

    // Segment c.
    assign c = C | ~B | A;

    // Segment d.
    assign d = D | (B & ~A) | (B & ~C) |
               (~C & ~A) | (C & ~B & A);

    // Segment e.
    assign e = (B & ~A) | (~C & ~A);

    // Segment f.
    assign f = D | (C & ~A) | (C & ~B) | (~A & ~B);

    // Segment g.
    assign g = D | (B & ~C) | (C & ~A) | (C & ~B);
endmodule

```

Testbench koji simulira konvertor za dozvoljene BCD ulaze je dat [OVDE](#).

c) Celokupnu funkcionalnost definisanu u okviru ove tačke zadatka moguće je dekomponovati na sledeće manje zahteve:

1. Kontrola uključenosti segmenata
2. Detekcija nedozvoljenog ulaza
3. Detekcija vodećih nula

Zahtev (1) podrazumeva uvođenje dodatnog signala u okviru kombinacione mreže BCD konvertora koji bi omogućio isključivanje svih segmenata LED displeja. Ovaj signal bi predstavljao ulazni signal u BCD konvertor dok bi logika za kontrolu ovog signala bila izmeštena izvan samog BCD konvertora. Funkcionalni zahtev (2) podrazumeva uvođenje još jednog dodatnog izlaznog signala koji ima za cilj prosleđivanje informacije o nevalidnosti ulaza ostalim BCD konvertorima koji čine višecifreni BCD konvertor. Jedan od najsloženijih funkcionalnih zahteva podrazumeva implementaciju logike za detekciju vodećih nula koja ima za cilj isključivanje segmenata onih LED displeja koji su povezani na vodeće nule. Za potpunu implementaciju ove logike neophodno je da do svakog BCD konvertora, u nizu konvertora koji čine višecifreni BCD konvertor, propagira informacija o tome da li su prethodni BCD konvertori viših cifara imali niz vodećih nula. Na taj način bi se omogućilo isključivanje segmenata BCD konvertora trenutne cifre ukoliko je i ona nula. Najniža cifra se izuzima iz ove kontrole da bi se za broj 0000 sačuvao prikaz jedne nule.

Na osnovu analize tri glavna funkcionalna zahteva, moguće je identifikovati sledeće signale u okviru svakog od BCD konvertora:

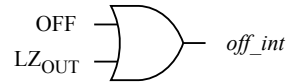
Tabela 6: Opis upravljačkih signala BCD konvertora

Naziv signala	Ulazni/Izlazni	Aktivni logički nivo	Opis
ERROR	Izlazni	1	Označava nedozvoljenu kombinaciju na podatkovnom ulazu tog bloka. Ne zavisi od signala OFF i LZ. Za globalnu grešku koriste se detekcije izvornih BCD podataka, pre nametanja slova E.
OFF	Ulazni	1	Kontroliše uključenost segmenata. Kada je postavljen na „1“, svi segmenti su isključeni, dok je u suprotnom stanje segmenata određeno drugim signalima.
LZ _{IN}	Ulazni	1	<i>Leading Zeros</i> ulazni signal BCD konvertora niže cifre prihvata <i>Leading Zeros</i> sa BCD konvertora više cifre. Jedinica znači da su sve više cifre nule; nula znači da je već naišla nenulta cifra.
LZ _{OUT}	Izlazni	1	<i>Leading Zeros</i> izlazni signal BCD konvertora više cifre se prosleđuje na BCD konvertor niže cifre. Jedinica znači da su trenutna i sve više cifre nule; inače je signal nula.

ERROR signal predstavlja izlazni signal iz jednog BCD konvertora koji se generiše kada se na ulazu pojavi binarni sadržaj u opsegu od 1010 -1111. Na osnovu analize sprovedene u tačkama a) i b) i uvidom u Karnoove karte svakog od izlaznih signala možemo izvesti minimalnu logičku funkciju *Error* signala:

$$ERROR = D \cdot (C + B) \quad (1.21)$$

Na putu do realizacije kompletne kontrole uključenosti svih segmenata LED displeja, uvešćemo jedan pomoćni signal `off_int` koji je aktivan u logičkoj jedinici. Logika za generisanje ovog internog signala BCD konvertora je prikazana na slici 17.



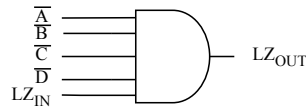
Slika 17: Logika za kontrolu off_int signala.

Do postavljanja ovog signala na nivo logičke jedinice dolazi ukoliko je eksterni signal **OFF** postavljen na nivo logičke jedinice ili ukoliko je **LZ_OUT** signal postavljen na nivo logičke jedinice. Drugim rečima, segment koji je kontrolisan **off_int** signalom će biti isključen ako se izvan BCD konvertora aktivira **OFF** signal ili ukoliko je BCD konvertor u nizu vodećih nula.

Logika za generisanje **LZ_OUT** signala se nalazi unutar BCD konvertora i definisana je izrazom (1.22) dok je implementacija predstavljena na slici 18.

$$LZ_{OUT} = LZ_{IN} \bar{D} \bar{C} \bar{B} \bar{A} \quad (1.22)$$

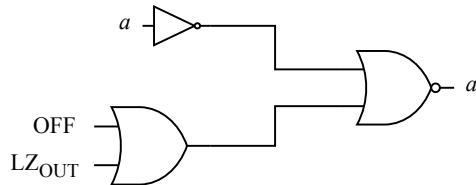
Dakle, ukoliko su u okviru BCD konvertora viših cifara detektovane vodeće nule ($LZ_{IN} = 1$) i ako se na ulazu trenutnog BCD konvertora nalazi binarna kombinacija koja odgovara cifri nula, **LZ_OUT** se postavlja na nivo logičke jedinice.



Slika 18: Logika za kontrolu LZ_OUT signala.

Logika ilustrovana na slikama 17 i 18 predstavlja glavne komponente kontrole uključenosti svakog od segmenata.

Na slici 19 je prikazana logika kojom se kontroliše uključenost segmenta a . Kao što se sa slike 19 može i uočiti, signal a' predstavlja dodatno kontrolisan signal a čija je funkcija već predstavljena u prethodnim tačkama.

Slika 19: Logika za kontrolu uključenosti segmenta a .

Na slici 20 proširena je logika samo segmenta a : $a' = a \overline{OFF + LZ_{OUT}}$. Izlazi dva NILI kola kombinuju se ILI kolom, pa završno NILI kolo daje negaciju njihovog zbira i signala za gašenje. Izlaz **ERROR** i segmenti $b'-g'$ nisu razrađeni na ovoj slici.

Za samostalni rad: Za ostale izlazne signale $b'-g'$ nacrtati odgovarajuće logičke šeme.

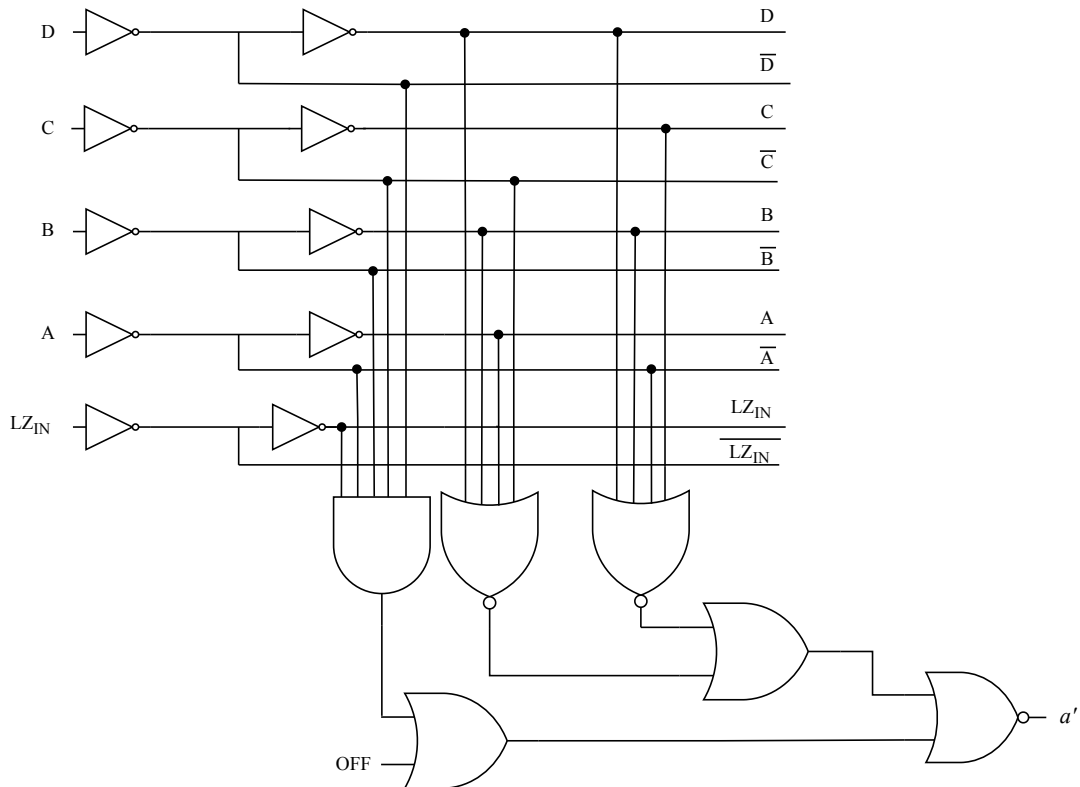
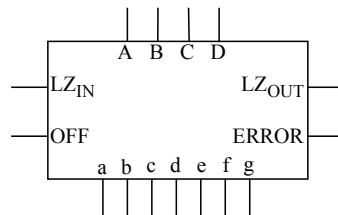
Logika segmenta a sa slike 20, zajedno sa logikom ostalih segmenata i signala **ERROR**, objedinjena je u funkcionalni blok BCD konvertora koji će biti iskorišćen za sintezu višecifrenog BCD konvertora. Ovaj funkcionalni blok je predstavljen na slici 21.

Za prikaz četiri cifre povezujemo četiri ovakva bloka. Detekcija greške svake izvorne cifre označena je sa R_i , gde indeks 0 predstavlja jedinice:

$$R_i = D_i(C_i + B_i), \quad i \in \{0, 1, 2, 3\}. \quad (1.23)$$

Globalna greška računa se ILI kolom 3:

$$G = R_3 + R_2 + R_1 + R_0. \quad (1.24)$$

Slika 20: Razrađena logika segmenta a sa kontrolom gašenja.

Slika 21: Funkcionalni blok BCD konvertora.

Više cifre dobijaju $OFF = G$. Njihovi podatkovni ulazi nisu izmenjeni, pa njihovi izlazi ERROR neposredno daju R_3 , R_2 i R_1 . Za jedinice posebno računamo R_0 iz izvornih bitova, pre ILI kola 1 i 2. Ta kola nameću:

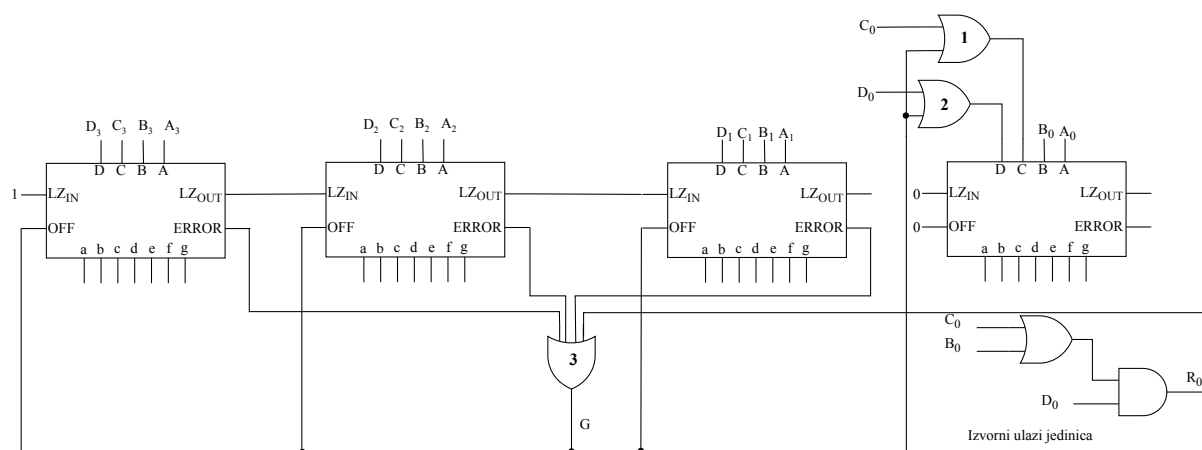
$$\begin{aligned} C'_0 &= C_0 + G, \\ D'_0 &= D_0 + G. \end{aligned} \quad (1.25)$$

Kada je $G = 1$, ulaz konvertora jedinica ima oblik $11xx$, pa prema tački a) prikazuje E. ERROR izlaz tog konvertora nije povezan sa kolom 3: on ispituje već izmenjeni podatak i ne sme da određuje globalnu grešku. Ponovljene oznake C_0 , B_0 i D_0 kod dodatnih kola na šemi označavaju iste izvorne ulaze jedinica.

Putanja ide od izvornih ulaza, preko detekcije greške, do upravljanja prikazom. Nema povratne putanje niti signala inicijalizacije. Posle promene 00A5 u 0504 prikaz se, nakon smirivanja kola, sam menja iz E u 504.

Signal LZ_{IN} najviše cifre vezan je na 1; LZ_{OUT} svake više cifre prosleđuje se sledećoj. Kod jedinica su LZ_{IN} i OFF vezani na 0. Tako se 0000 prikazuje kao jedna nula, a unutrašnje nule, poput nule u 0504, ostaju vidljive. Izlaz LZ_{OUT} cifre 1 i oba upravljačka izlaza jedinica nisu potrebni daljim blokovima.

Pomoćni modul u nastavku proširuje NILI mrežu iz tačke a). Signal `off_int` dodat je završnom kolu svakog segmenta, dok ERROR ostaje nezavisan od gašenja.



Slika 22: Četvorocifreni kombinacioni BCD konverter: greška se detektuje pre nametanja slova E jedinicama.

Verilog kod koji implementira konvertor jedne cifre sa detekcijom greške i vodeće nule. Kod je grupisan po segmentima.

```

module bcd_cifra (
    input logic D, C, B, A,
    input logic OFF, LZ_IN,
    output logic a, b, c, d, e, f, g,
    output logic ERROR, LZ_OUT
);
    logic off_int;
    logic D_n, C_n, B_n, A_n;
    logic a1, a2;
    logic b1, b2, b3, b4;
    logic c1, d1;
    logic e1, e2, e3;
    logic f1, f2, f3;
    logic g1;

    // Detekcija greske ne zavisi od signala za gasenje.
    assign ERROR = D & (C | B);
    assign LZ_OUT = LZ_IN & ~D & ~C & ~B & ~A;
    assign off_int = OFF | LZ_OUT;

    // Invertori: NILI kola sa vezanim ulazima.
    assign D_n = ~(D | D);
    assign C_n = ~(C | C);
    assign B_n = ~(B | B);
    assign A_n = ~(A | A);

    // Segment a sa kontrolom gasenja.
    assign a1 = ~(D | C_n | B | A);
    assign a2 = ~(D | C | B | A_n);
    assign a = ~(a1 | a2 | off_int);

    // Segment b; b1 i b2 koriste se i za segment c.
    assign b1 = ~(D_n | B_n);
    assign b2 = ~(D_n | C_n);
    assign b3 = ~(C_n | B_n | A);
    assign b4 = ~(C_n | B | A_n);
    assign b = ~(b1 | b2 | b3 | b4 | off_int);

    // Segment c.
    assign c1 = ~(C | B_n | A);
    assign c = ~(b1 | b2 | c1 | off_int);

    // Segment d; zajednicki clanovi sa segmentom a.
    assign d1 = ~(D | C_n | B_n | A_n);
    assign d = ~(a1 | a2 | d1 | off_int);

    // Segment e.
    assign e1 = ~(D | A_n);
    assign e2 = ~(C | B | A_n);
    assign e3 = ~(D | C_n | B);
    assign e = ~(e1 | e2 | e3 | off_int);

    // Segment f.
    assign f1 = ~(D | C | A_n);
    assign f2 = ~(D | C | B_n);
    assign f3 = ~(D | B_n | A_n);
    assign f = ~(f1 | f2 | f3 | off_int);

    // Segment g; zajednicki clan sa segmentom d.
    assign g1 = ~(D | C | B);
    assign g = ~(g1 | d1 | off_int);
endmodule

```

Testbench koji simulira konvertor jedne cifre je dat [OVDE](#).

Glavni modul eksplicitno povezuje četiri instance pomoćnog modula. Ulazi BCD3–BCD0 imaju redosled bitova *DCBA*, a izlazi SEG3–SEG0 redosled *abcdefg* od bita 6 do bita 0. Instanca CIFRA0 predstavlja jedinice; prazni priključci `.ERROR()` i `.LZ_OUT()` označavaju izlaze koji se ne koriste. Verilog kod koji implementira četvorocifreni konvertor sa slike 22. Za simulaciju koristiti i prethodni pomoćni modul `bcd_cifra.sv`.

```

module zadatak (
    input logic [3:0] BCD3, BCD2, BCD1, BCD0,
    output logic [6:0] SEG3, SEG2, SEG1, SEG0
);
    logic R3, R2, R1, R0, G;
    logic LZ3, LZ2;
    logic CO_mod, DO_mod;

    // Greska jedinica: izvorni ulaz, pre nametanja slova E.
    assign R0 = BCD0[3] & (BCD0[2] | BCD0[1]);
    assign G = R3 | R2 | R1 | R0;

    // Ako postoji greska, jedinice dobijaju kod 11xx (E).
    assign CO_mod = BCD0[2] | G;
    assign DO_mod = BCD0[3] | G;

    // SEG[6:0] = {a, b, c, d, e, f, g}; BCD[3:0] = DCBA.
    bcd_cifra CIFRA3 (
        .D(BCD3[3]), .C(BCD3[2]), .B(BCD3[1]), .A(BCD3[0]),
        .OFF(G), .LZ_IN(1'b1), .ERROR(R3), .LZ_OUT(LZ3),
        .a(SEG3[6]), .b(SEG3[5]), .c(SEG3[4]), .d(SEG3[3]),
        .e(SEG3[2]), .f(SEG3[1]), .g(SEG3[0])
    );

    bcd_cifra CIFRA2 (
        .D(BCD2[3]), .C(BCD2[2]), .B(BCD2[1]), .A(BCD2[0]),
        .OFF(G), .LZ_IN(LZ3), .ERROR(R2), .LZ_OUT(LZ2),
        .a(SEG2[6]), .b(SEG2[5]), .c(SEG2[4]), .d(SEG2[3]),
        .e(SEG2[2]), .f(SEG2[1]), .g(SEG2[0])
    );

    // LZ_OUT cifre 1 nije potreban: jedinice se ne gase zbog nule.
    bcd_cifra CIFRA1 (
        .D(BCD1[3]), .C(BCD1[2]), .B(BCD1[1]), .A(BCD1[0]),
        .OFF(G), .LZ_IN(LZ2), .ERROR(R1), .LZ_OUT(),
        .a(SEG1[6]), .b(SEG1[5]), .c(SEG1[4]), .d(SEG1[3]),
        .e(SEG1[2]), .f(SEG1[1]), .g(SEG1[0])
    );

    // ERROR sa izmenjenog ulaza jedinica ne ulazi u racunanje G.
    bcd_cifra CIFRA0 (
        .D(DO_mod), .C(CO_mod), .B(BCD0[1]), .A(BCD0[0]),
        .OFF(1'b0), .LZ_IN(1'b0), .ERROR(), .LZ_OUT(),
        .a(SEG0[6]), .b(SEG0[5]), .c(SEG0[4]), .d(SEG0[3]),
        .e(SEG0[2]), .f(SEG0[1]), .g(SEG0[0])
    );
endmodule

```

Testbench koji simulira konvertor sa slike 22 je dat [OVDE](#).

2 Zadaci za samostalni rad

2.1 Zadatak 1

- a) Pomoću dekodera $n/2^n$ i $m/2^m$ projektovati dekodер $(n+m)/2^{n+m}$. Na raspolaganju su dvoulazna logička I kola. Dekodere $n/2^n$ i $m/2^m$ crtati kao blokove.
- b) Pomoću dvoulaznih I kola i invertora realizovati dekodер 4/16 kod koga su i ulazni i izlazni signali aktivni na logičkoj jedinici. Težiti da broj upotrebljenih logičkih kola bude minimalan.
- c) Projektovati kolo dekodera koje ima 13 ulaza tako da realizacija bude izvedena u maksimalno 4 nivoa. Na raspolaganju su dvoulazna I kola i invertori. Težiti da broj upotrebljenih logičkih kola bude minimalan. Vreme propagiranja kroz logičko kolo iznosi t_{dlk} dok se kašnjenje signala invertora može zanemariti.

2.2 Zadatak 2

Funkciju:

$$Y = \bar{D}\bar{C}\bar{A} + \bar{B}A + \bar{C}A \quad (2.1)$$

realizovati isključivo korišćenjem osnovnih logičkih kola sa proizvoljnim brojem ulaza i multipleksa 4/1 za čije selekzione signale S_1S_0 važi:

- a) $S_1S_0 = DC$
- b) $S_1S_0 = BA$
- c) $S_1S_0 = DB$
- d) $S_1S_0 = CA$

Koja od realizacija a), b), c) i d) zahteva minimalan broj logičkih kola?

Uslov poredenja: Navesti koje tipove kola smatrate osnovnim i da li se zajednički međurezultati dele između ulaza multipleksa. Na primer, može se porediti broj NE, I i ILI kola, uz besplatne konstante i vodove. Bez dogovorene biblioteke i merila troška broj kola nije jednoznačno određen.